

# BetaZero: MCTS With Learned Backup

Design Document

David Shin

April 30, 2026

## Contents

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Motivation</b>                                                  | <b>2</b>  |
| <b>2</b> | <b>Related Work</b>                                                | <b>2</b>  |
| <b>3</b> | <b>Background</b>                                                  | <b>3</b>  |
| 3.1      | NNUE . . . . .                                                     | 3         |
| 3.2      | KataGo’s Uncertainty Head . . . . .                                | 3         |
| <b>4</b> | <b>Architecture Overview</b>                                       | <b>4</b>  |
| 4.1      | Network Structure . . . . .                                        | 4         |
| 4.2      | Latent Projections ( $\mathbf{z}_s$ and $\mathbf{z}_a$ ) . . . . . | 5         |
| 4.3      | Backup Network . . . . .                                           | 5         |
| <b>5</b> | <b>Training Targets</b>                                            | <b>7</b>  |
| 5.1      | Targets for $V$ , $U$ , $Q$ , and $W$ . . . . .                    | 7         |
| 5.2      | Targets for $AV$ and $AU$ . . . . .                                | 8         |
| <b>6</b> | <b>Training Procedure</b>                                          | <b>9</b>  |
| 6.1      | Sampling for $\mathcal{A}_\phi$ . . . . .                          | 9         |
| 6.2      | Bootstrapping and Regularization . . . . .                         | 9         |
| <b>7</b> | <b>Inference and Search Integration</b>                            | <b>9</b>  |
| 7.1      | NNUE-Based Backup . . . . .                                        | 9         |
| 7.2      | Use in PUCT . . . . .                                              | 10        |
| 7.3      | Future Direction: Replacing PUCT . . . . .                         | 10        |
| <b>8</b> | <b>Spatial Action Latents</b>                                      | <b>11</b> |
| <b>9</b> | <b>Hyperparameters</b>                                             | <b>11</b> |

## 1. Motivation

AlphaZero aggregates child value estimates into a parent estimate using a visit-count-weighted average:

$$Q(p) = \frac{V(p) + \sum_i N_i Q_i}{1 + \sum_i N_i}.$$

This is a reasonable heuristic but has well-known theoretical weaknesses. It conflates *exploratory* visits (induced by PUCT’s exploration term and auxiliary mechanisms like policy temperature and Dirichlet noise) with *confirmatory* visits (made because a child is believed to be good). A child that was heavily explored but ultimately found to be poor will still exert downward pressure on  $Q(p)$  proportional to its visit count. Additionally, a child suddenly found to be very good needs many more visits to offset highly-visited-but-inferior siblings than should be theoretically necessary.

Experienced go and chess players intuitively understand the suboptimality of estimating a parent node as an average over its children. Game theoretically, the value of a position is determined by the opponent’s **best** response, not by an average over possible responses.

BetaZero replaces this hand-crafted backup with a **learned backup network**  $\mathcal{A}_\phi$  that takes children statistics as input and outputs a posterior parent estimate  $Q(p)$ . The hope is that  $\mathcal{A}_\phi$  learns to discount exploratory visits and produce better-calibrated parent estimates, which in turn improves the quality of search.

## 2. Related Work

Guez et al. (2018) proposed MCTSnets<sup>1</sup>, an architecture that replaces **all** components of MCTS — selection, evaluation, and backup — with learned neural network functions, trained end-to-end via gradient-based optimization. Each node in the search tree stores a learned vector-valued memory, and a backup network updates parent memory based on child memory along the traversed simulation path. MCTSnets was demonstrated on Sokoban with 25 simulations, significantly outperforming standard MCTS baselines.

While MCTSnets validates the idea that learned backup can outperform hand-crafted backup, its fully end-to-end design has two significant drawbacks. First, replacing selection, evaluation, and backup simultaneously provides too many degrees of freedom and too weak a learning signal to scale practically. The training signal (task reward) must drive learning across all components at once, with no intermediate supervision. The vector-valued node statistics are opaque, making the system difficult to reason about, debug, or regularize. Second, MCTSnets suffers from a computational issue: even if the backup network could be well trained, evaluating a neural network at every backup step makes the overall system impractical to use in competitive settings, where millions of backup steps are performed per move.

BetaZero can be viewed as a more constrained version of this idea. It retains AlphaZero’s selection (PUCT) and evaluation (value network) intact, and focuses specifically on improv-

---

<sup>1</sup>A. Guez, T. Weber, I. Antonoglou, K. Simonyan, O. Vinyals, D. Wierstra, R. Munos, D. Silver. *Learning to Search with MCTSnets*. ICML 2018. <https://arxiv.org/abs/1802.04697>

ing the backup step. The backup network’s outputs are interpretable scalars ( $Q$  and  $W$ ) with well-defined training targets (game outcomes and uncertainty estimates). This allows for more precise learning with intermediate supervision, rather than relying on end-to-end credit assignment through the entire search computation graph. The computational issue is addressed by adopting NNUE mechanics (see Section 3.1).

### 3. Background

#### 3.1. NNUE

The Efficiently Updatable Neural Network (NNUE)<sup>2</sup> was introduced by Yu Nasu in 2018 for use in computer shogi engines. The key insight is that when a neural network’s input changes only sparsely between evaluations, the first layer’s output can be maintained incrementally rather than recomputed from scratch.

In the standard NNUE design, the first layer maps a large, sparse input (encoding the board position) to a hidden representation called the **accumulator**. When a move is made, only a few input features change (a piece leaves one square and arrives at another). Rather than recomputing the full first-layer matrix-vector product, the accumulator is updated via a **subtract-add** operation: subtract the weight columns corresponding to removed features and add the columns corresponding to added features. The subsequent layers are small and recomputed from scratch using the updated accumulator. This makes the per-evaluation cost independent of the total input dimensionality, depending only on the number of changed features.

NNUE requires very specific structural conditions: a large, sparse first layer whose inputs change incrementally, followed by small dense layers. These conditions have limited its adoption to a narrow set of games; it has been successfully applied to shogi and chess, but has not been adapted to go. As we shall see, BetaZero’s backup network has been carefully architected to satisfy these conditions.

#### 3.2. KataGo’s Uncertainty Head

KataGo<sup>3</sup> introduces an **uncertainty head**  $U$  alongside the standard value and policy heads. This head predicts how much the value estimate at a given position is expected to shift over the remainder of the game, trained against the target:

$$U(p)_{\text{target}} = \left( V(p) - (1 - \lambda) \sum_{t \geq T} Q_t \lambda^{t-T} \right)^2,$$

where  $T$  is the current turn,  $Q_t$  is the final search  $Q$  at the root at the end of turn  $t$ , and  $\lambda$  is an exponential decay parameter (KataGo uses  $\lambda = 5/6$ ). KataGo uses  $U$  to weight visits during backup: nodes with higher uncertainty receive proportionally more search effort.

---

<sup>2</sup>Y. Nasu. *Efficiently Updatable Neural-Network-based Evaluation Functions for Computer Shogi*. Ziosoft Computer Shogi Club, 2018. <https://www.chessprogramming.org/NNUE>

<sup>3</sup>D. Wu. *Accelerating Self-Play Learning in Go*. 2019. <https://arxiv.org/abs/1902.10565>. See also: <https://github.com/lightvector/KataGo/blob/master/docs/KataGoMethods.md#uncertainty-weighted-mcts-playouts>

BetaZero adopts this uncertainty head, but repurposes it: rather than using  $U$  to weight visits, we provide it as an input to the backup network  $\mathcal{A}_\phi$ , allowing the network to condition its backup strategy on the uncertainty of each child.

## 4. Architecture Overview

### 4.1. Network Structure

During training, there is a single neural network comprising a ResNet trunk, several output heads, and a backup subnetwork. Figure 1 shows the overall structure.

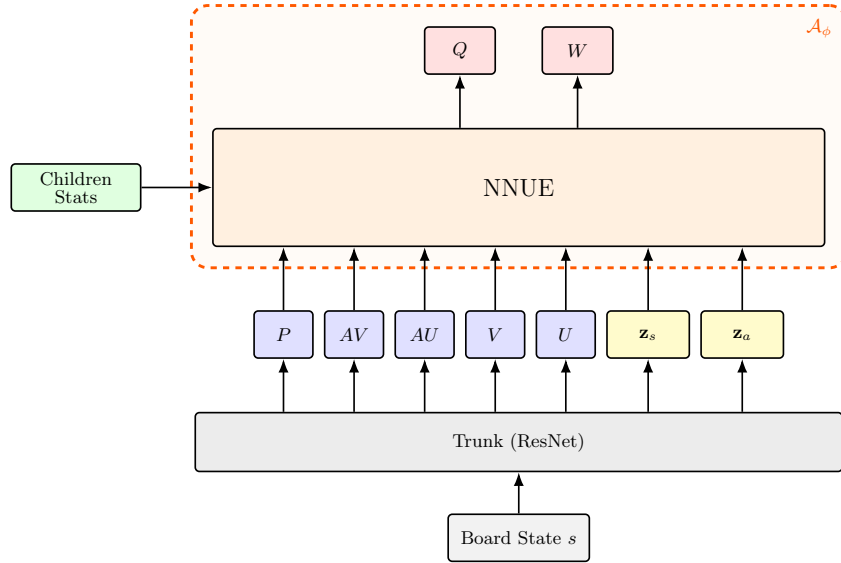


Figure 1: Network architecture. The trunk feeds into heads  $P$ ,  $AV$ ,  $AU$ ,  $V$ ,  $U$ , and two latent projections  $\mathbf{z}_s$  and  $\mathbf{z}_a$ , all of which feed into the NNUE-based backup subnetwork  $\mathcal{A}_\phi$  alongside dynamic children statistics from the search tree.  $\mathcal{A}_\phi$  outputs  $Q$  (posterior value) and  $W$  (posterior uncertainty). At export time,  $\mathcal{A}_\phi$  is copied out for CPU-side incremental evaluation via NNUE mechanics.

The trunk produces features that branch into:

- **Policy head  $P$** : prior move probabilities (same as AlphaZero).
- **Action-value head  $AV$** : per-action value predictions (see Section 5.2).
- **Action-uncertainty head  $AU$** : per-action uncertainty predictions (see Section 5.2).
- **Value head  $V$** : prior scalar value estimate (same as AlphaZero).
- **Uncertainty head  $U$** : prior scalar uncertainty estimate (adapted from KataGo; see Section 3.2).
- **Static latent  $\mathbf{z}_s$** : a flat vector capturing global positional properties (see Section 4.2).
- **Action latent  $\mathbf{z}_a$** : a per-action representation capturing positional context for each action (see Section 4.2).

These outputs, together with dynamic children statistics from the search tree, feed into the

backup subnetwork  $\mathcal{A}_\phi$ , which outputs:

- $Q$ : posterior value estimate.
- $W$ : posterior uncertainty estimate.

At expansion time, the entire network is evaluated on the GPU in a single forward pass, with children statistics set to zero. This produces all head outputs ( $P$ ,  $AV$ ,  $AU$ ,  $V$ ,  $U$ ,  $Q$ ,  $W$ ) and the intermediate activations of  $\mathcal{A}_\phi$  (the latent vectors and the NNUE accumulator state). These intermediates are stored at the node to enable incremental CPU-side evaluation on subsequent backups. The initial values of  $Q^*$  and  $W^*$  (the LoTE and LoTV baselines; see Section 4.3) are set to  $V$  and  $U$  respectively.

At export time, the  $\mathcal{A}_\phi$  portion of the network is copied out as a separate lightweight network. On each backup, only  $\mathcal{A}_\phi$  is re-evaluated on the CPU using NNUE mechanics (see Section 7.1), while the trunk and other heads are not re-run.

## 4.2. Latent Projections ( $\mathbf{z}_s$ and $\mathbf{z}_a$ )

The backup network conditions on positional context from the trunk. If the full trunk features were stored at each node, the memory cost would be prohibitive. To address this, two compact **projections** are used:

**Static latent  $\mathbf{z}_s$ :** a flat vector of fixed dimension  $d_s$  (e.g., 32–64), capturing global positional properties such as tactical sharpness or game phase.  $\mathbf{z}_s$  enters the backup network after the accumulator layer, providing the same global context regardless of which child is being considered.

**Action latent  $\mathbf{z}_a$ :** a per-action representation that provides positional context specific to each child action.  $\mathbf{z}_a$  enters the backup network at the per-child embedding layer (layer 1), enriching each child’s embedding with the positional meaning of its action. See Section 8 for further discussion of how  $\mathbf{z}_a$  can encode correlation beliefs between actions.

For games with spatially structured action spaces (Go, Hex, Othello),  $\mathbf{z}_a$  is naturally a spatial tensor of shape  $(H, W, r)$ . For games like Chess,  $\mathbf{z}_a$  may be omitted initially in favor of  $\mathbf{z}_s$  alone.

## 4.3. Backup Network

The backup network  $\mathcal{A}_\phi$  is a lightweight network evaluated at every *backup step*. Its outputs are stored at the node and read during selection (see Section 7.2). The outputs for a node  $p$  are:

- $Q(p)$ : posterior value estimate for the parent node.
- $W(p)$ : posterior uncertainty estimate for the parent node.

Figure 2 shows the internal structure of  $\mathcal{A}_\phi$ .

The inputs to  $\mathcal{A}_\phi$  for a node  $p$  with children  $\{c_1, \dots, c_k\}$  fall into two categories, corresponding to the two zones in Figure 2:

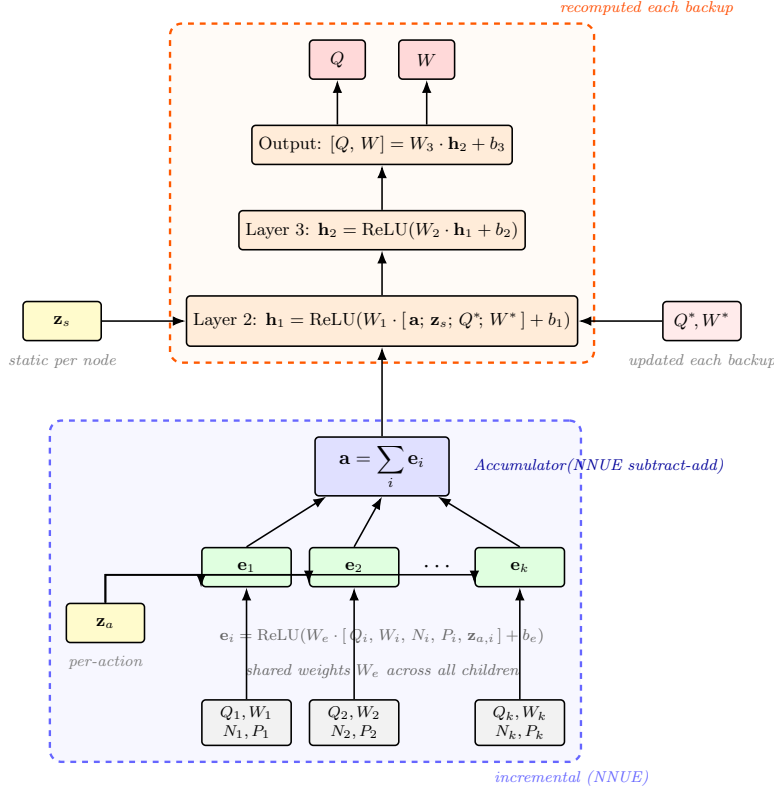


Figure 2: Internal structure of  $\mathcal{A}_\phi$ . The first layer computes a per-child embedding  $\mathbf{e}_i$  using **shared weights**  $W_e$  across all children, incorporating the per-action latent  $\mathbf{z}_{a,i}$ . The embeddings are sum-pooled into an accumulator  $\mathbf{a}$ , which supports NNUE-style subtract-add updates. The subsequent layers receive the accumulator alongside the static latent  $\mathbf{z}_s$  (static per node) and the LoTE/LoTV baselines  $Q^*, W^*$  (updated each backup but computed outside the network), then produce the final  $Q$  and  $W$  outputs.

**Per-child inputs (accumulator layer).** Each child  $c_i$  contributes a tuple  $(Q_i, W_i, N_i, P_i)$ . For expanded children ( $N_i > 0$ ),  $Q_i$  and  $W_i$  are outputs of  $\mathcal{A}_\phi$  previously computed at child  $c_i$ . For unexpanded children ( $N_i = 0$ ),  $Q_i = AV_i$  and  $W_i = AU_i$ , the per-action predictions from the main network (see Section 5.2). Each child’s tuple is embedded through a shared linear transform that also incorporates the per-action latent:

$$\mathbf{e}_i = \text{ReLU}(W_e \cdot [Q_i, W_i, N_i, P_i, \mathbf{z}_{a,i}] + b_e),$$

where  $W_e$  and  $b_e$  are the same for all children, and  $\mathbf{z}_{a,i}$  is the slice of  $\mathbf{z}_a$  corresponding to action  $i$ . This weight-sharing is a key difference from standard NNUE (e.g., in Stockfish), where each input feature has independent weights. Here, every child is semantically homogeneous —  $(Q, W, N, P)$  means the same thing regardless of which action it corresponds to — so shared weights are natural and dramatically reduce the parameter count. The per-action context  $\mathbf{z}_{a,i}$  breaks the symmetry, allowing the network to condition each child’s embedding on the positional meaning of its action.

The embeddings are sum-pooled into an **accumulator**:

$$\mathbf{a} = \sum_i \mathbf{e}_i.$$

**Context inputs (subsequent layers).** The accumulator  $\mathbf{a}$  is concatenated with:

- The static latent  $\mathbf{z}_s(p)$  (static per node).
- The LoTE and LoTV baselines  $Q^*(p)$  and  $W^*(p)$  (updated each backup, computed outside the network).

The baselines are defined as:

$$Q^*(p) = \frac{V(p) + \sum_i N_i Q_i}{1 + \sum_i N_i}$$

$$W^*(p) = \frac{U(p) + (V(p) - Q^*(p))^2 + \sum_i N_i (W_i + (Q_i - Q^*(p))^2)}{1 + \sum_i N_i}$$

where  $V(p)$  and  $U(p)$  are the prior value and uncertainty from the trunk. At expansion time,  $Q^* = V$  and  $W^* = U$ . The baselines are maintained incrementally from running sums and updated outside the network before each  $\mathcal{A}_\phi$  evaluation.

Providing  $Q^*$  and  $W^*$  as direct inputs (rather than requiring the network to derive them from raw children statistics) serves two purposes. First, it makes the LoTE regularization trivially representable: to match the baseline, the network simply passes  $Q^*$  through to the output. Second, because  $Q^*$  and  $W^*$  enter after the accumulator layer, they do not affect the NNUE subtract-add — only the subsequent layers, which are recomputed from scratch each backup anyway.

The LoTV baseline  $W^*(p)$  is motivated by interpreting  $W$  as a variance. This interpretation is loosely justified: the  $W$  target is defined as a squared difference between  $Q$  values (see Section 5.1), which matches the structure of variance computation. The interpretation need not be rigorous —  $W^*(p)$  serves only as an input suggestion that  $\mathcal{A}_\phi$  is free to use or ignore.

Because  $\mathcal{A}_\phi$  is called at every backup step (typically  $O(n \log n)$  times in a search tree with  $n$  nodes), it must be small.

## 5. Training Targets

### 5.1. Targets for $V$ , $U$ , $Q$ , and $W$

$V(p)$  and  $U(p)$  are scalar outputs of the trunk, representing the **prior** value and uncertainty estimates conditioned only on the board state.  $Q(p)$  and  $W(p)$  are outputs of  $\mathcal{A}_\phi$ , representing the **posterior** estimates additionally conditioned on children statistics from search.

**$V$  and  $Q$  targets.** Both are trained against the game outcome  $Z$ .  $V$  learns to predict  $Z$  from the board state alone.  $Q$  learns to predict  $Z$  with the additional benefit of search evidence, and can in principle recover AlphaZero’s backup by learning to output  $Q^*$ .

**$U$  and  $W$  targets.** Both predict uncertainty: how much the value estimate is expected to swing over the remainder of the game. We adapt KataGo’s uncertainty target (Section 3.2), with one modification.

For  $U$  (the prior), the target at turn  $T$  is:

$$U(p)_{\text{target}} = \left( V(p) - (1 - \lambda) \sum_{t>T} Q_t \lambda^{t-1-T} \right)^2,$$

where  $Q_t$  represents the final  $Q$  at the tree root at the end of turn  $t$ , and  $\lambda$  is a system hyperparameter.

For  $W$  (the posterior), the target is analogous. Denoting by  $Q^{(n)}(p)$  the output of  $\mathcal{A}_\phi$  after  $n$  search iterations:

$$W(p)_{\text{target}} = \left( Q^{(n)}(p) - (1 - \lambda) \sum_{t>T} Q_t \lambda^{t-1-T} \right)^2.$$

Note that both sums range over  $t > T$ , whereas KataGo’s sum ranges over  $t \geq T$ . The reason is that the expected difference between the current  $Q^{(n)}(p)$  and  $Q_T$  depends on  $n$ , the search iteration at which the sample was obtained. Including  $t = T$  would introduce an inappropriate coupling between  $n$  and  $W(p)_{\text{target}}$ .

The critical grounding property of the  $W$  target is that  $W(p) = 0$  for any position whose game-theoretic value is fully resolved — either because it directly precedes a terminal state, or because its subtree has been exhaustively searched (e.g., proven mate, proven draw). This gives the backup network a reliable anchor:  $W_i = 0$  means child  $c_i$ ’s value  $Q_i$  is completely trustworthy, and  $W_i = \epsilon$  means it is nearly so.

As noted in Section 4.3, we interpret  $W$  as a variance to compute a LoTV baseline over children statistics. This is merely a chosen *interpretation* and is not rigorously justified. It need not be rigorously justified, as the LoTV-computed value serves merely as a *suggestion* to  $\mathcal{A}_\phi$  on how to compute the actual uncertainty backup, which  $\mathcal{A}_\phi$  is free to use or ignore. Whether the target definition or the LoTV baseline are useful are empirical questions; what ultimately matters is whether the choices help  $\mathcal{A}_\phi$  produce better  $Q(p)$  estimates.

## 5.2. Targets for $AV$ and $AU$

The main network includes per-action heads  $AV(s, a)$  and  $AU(s, a)$  that predict the value and uncertainty at the child state reached by action  $a$ . Their training targets are the network’s own prior evaluations at those child states:

$$\begin{aligned} AV(s, a)_{\text{target}} &= V(s'), \\ AU(s, a)_{\text{target}} &= U(s'), \end{aligned}$$

where  $s'$  is the state reached by playing action  $a$  from  $s$ .

To produce these targets, children of sampled root nodes are expanded during training. This can be done exhaustively (expanding all children) or by sampling a random subset and computing loss only on that subset.

$AV$  and  $AU$  serve as the default  $Q_i$  and  $W_i$  inputs for unexpanded children when feeding into  $\mathcal{A}_\phi$ . This allows the backup network to reason about all children — not just those that have been visited — using the network’s prior predictions as stand-ins.

Note that AlphaZeroArcade<sup>4</sup> already uses an  $AV$  head, whose output is used for the  $Q$  term in the PUCT formula for unvisited children.

## 6. Training Procedure

### 6.1. Sampling for $\mathcal{A}_\phi$

Following KataGo’s Playout Cap Randomization, some fraction of root nodes are sampled for NN training. Of these, a configurable fraction  $\alpha$  is allocated to training the network in the **node-expansion regime** (prior heads:  $P$ ,  $AV$ ,  $AU$ ,  $V$ ,  $U$ ), and the remainder  $1 - \alpha$  to training the network in the **node-backup regime** ( $Q$  and  $W$  conditioned on search evidence). The split ratio  $\alpha$  is a system hyperparameter; since the trunk and expansion-regime heads are larger and learn a harder function,  $\alpha > 0.5$  is likely appropriate.

For positions allocated to the backup regime, if  $N$  is the search budget in iterations, we uniformly sample some  $n$  from  $\{1, \dots, N\}$  and snapshot the tree statistics after  $n$  iterations have been performed. This means  $\mathcal{A}_\phi$  learns to produce a useful posterior estimate given any amount of search evidence, from very little ( $n$  small) to nearly the full budget ( $n \approx N$ ).

### 6.2. Bootstrapping and Regularization

Early in training,  $\mathcal{A}_\phi$  is uninformed, which risks destabilizing the feedback loop between search quality and training targets. To mitigate this, the network is regularized during early training to match the hand-crafted baselines:

$$\begin{aligned} Q(p)_{\text{baseline}} &= Q^*(p), \\ W(p)_{\text{baseline}} &= W^*(p). \end{aligned}$$

This regularization is annealed away as training progresses. Because  $Q^*(p)$  and  $W^*(p)$  are direct inputs to  $\mathcal{A}_\phi$  (injected after the accumulator; see Section 4.3), the baseline is trivially representable: the network simply learns to pass these inputs through to the output. This provides strong initial stability. As the annealing tapers, the network should gradually mature to produce more nuanced outputs informed by the accumulator,  $\mathbf{z}_s$ , and  $\mathbf{z}_a$ .

## 7. Inference and Search Integration

### 7.1. NNUE-Based Backup

The full network is evaluated once per node expansion on the GPU. This produces  $P$ ,  $AV$ ,  $AU$ ,  $V$ ,  $U$ ,  $Q$ ,  $W$ , and the intermediate activations of  $\mathcal{A}_\phi$  (the  $\mathbf{z}_s$  and  $\mathbf{z}_a$  vectors and the

---

<sup>4</sup><https://github.com/shindavid/AlphaZeroArcade>

NNUE accumulator state) — all stored at the node. The baselines  $Q^*$  and  $W^*$  are initialized to  $V$  and  $U$ .

On each subsequent backup, only  $\mathcal{A}_\phi$  is re-evaluated on the CPU using NNUE mechanics. Between consecutive backups at a given node, typically only one child’s statistics change. The accumulator is updated via a subtract-add operation — subtracting the old child embedding and adding the new one — rather than recomputing the full sum over all children. The  $\mathbf{z}_{a,i}$  values used in the per-child embedding are static (computed at expansion) and do not need to be recomputed. The baselines  $Q^*$  and  $W^*$  are updated from incrementally maintained running sums. The subsequent layers are then recomputed from the updated accumulator,  $\mathbf{z}_s$ ,  $Q^*$ , and  $W^*$ .

This makes the per-backup cost of  $\mathcal{A}_\phi$  independent of the branching factor, which is significant for games with large action spaces. The accumulator is initialized as part of the GPU forward pass at expansion time, so no separate  $O(k)$  initialization is needed on the CPU.

In self-play mode, evaluations of  $\mathcal{A}_\phi$  across simultaneous games are batched, amortizing cost. In single-game mode, we may be able to adopt a similar mechanism to standard MCTS parallelization, which involves multiple search threads and virtual visits. Some details remain to be worked out.

## 7.2. Use in PUCT

$\mathcal{A}_\phi$ ’s stored  $Q_i$  output for each child  $c_i$  is used directly in the PUCT selection formula:

$$\text{PUCT}_i = Q_i + c \cdot P_i \cdot \frac{\sqrt{1 + N(p)}}{1 + N_i}.$$

## 7.3. Future Direction: Replacing PUCT

Later, we may add a head to  $\mathcal{A}_\phi$  that outputs a score that replaces PUCT entirely. The challenge is devising a training target that captures the purpose of PUCT.

One idea for this is to define a notion of “exploration worthiness”. PUCT aims to identify actions worth exploring. What makes an action worth exploring? One candidate definition is to say that an action is worth exploring if doing so has strong potential for changing our current  $Q$  estimate. This suggests using a training target that is based on the change in  $Q(p)$  induced by visiting a given child. A similar alternative is to say that an action is worth exploring if *not* doing so for the remaining search budget is likely to produce a different  $Q$  estimate than an unconstrained usage of the remaining search budget. Still another possibility is to locate exploration worthiness in a likely reduction of  $W(p)$ , rather than an expected delta in  $Q(p)$ . It may be possible to produce targets corresponding to these notions.

All these options have the property that we can use an exact score of 0 for fully resolved positions (e.g., proven mate, proven draw), while a NN estimated position can be made to return a strictly positive score. This has the nice property that the selection criterion would naturally avoid re-exploring fully resolved positions, a property that standard PUCT lacks.

If something can be found to work here, it may be possible to remove one or more heads among  $P$ ,  $AU$ , and  $AV$ , as they exist primarily to facilitate the PUCT criterion.

## 8. Spatial Action Latents

The action latent  $\mathbf{z}_a$  provides per-action context to the backup network’s first layer. For games with spatially structured action spaces, it is naturally represented as a spatial tensor of shape  $(H, W, r)$ , where  $H \times W$  matches the board dimensions and  $r$  is a small channel depth (e.g., 4–8). The slice  $\mathbf{z}_{a,i}$  for action  $i$  is then an  $r$ -dimensional vector.

A spatial  $\mathbf{z}_a$  enables  $\mathcal{A}_\phi$  to represent **correlation beliefs between actions**. For example, in Go on a  $19 \times 19$  board, a spatial projection of shape  $(19, 19, r)$  could learn to map each candidate move to a vector on an  $r$ -dimensional hypersphere, with the cosine similarity between two move-vectors encoding the backup network’s belief about how correlated their values are. This would allow  $\mathcal{A}_\phi$  to reason about questions like: “move  $A$  and move  $B$  both attack the same group, so if  $A$ ’s value dropped after search,  $B$ ’s value likely dropped too.” The static latent  $\mathbf{z}_s$  alone cannot represent such per-action structure.

Note, however, that because the per-child embeddings are sum-pooled, the accumulator does not retain pairwise information between children.  $\mathbf{z}_a$  enriches each child’s embedding individually, but after pooling, the identity of which embedding came from which child is lost. The correlation structure is thus represented implicitly in the embedding space (similar actions produce similar embeddings that shift the accumulator in similar directions) rather than as explicit pairwise interactions. Whether this implicit representation is sufficient for the backup network to learn useful correlation-aware behavior is an empirical question.

The memory cost of a spatial  $\mathbf{z}_a$  is  $(H \times W \times r)$  floats per node. For Go with  $r = 8$ , this is  $19 \times 19 \times 8 = 2,888$  floats — reasonable for practical use.

This design is more natural for games with spatially structured action spaces (Go, Hex, Othello) than for games like Chess, where the action space is a set of (source, destination) pairs rather than single board intersections. For Chess,  $\mathbf{z}_a$  may be omitted initially, relying on  $\mathbf{z}_s$  alone.  $P_i$  already provides some per-action context (the prior probability), which partially compensates for the absence of  $\mathbf{z}_a$ .

## 9. Hyperparameters

Table 1 summarizes the tunable hyperparameters of the system.

| Symbol    | Description                                                      | Default / Suggested |
|-----------|------------------------------------------------------------------|---------------------|
| $\alpha$  | Fraction of sampled roots allocated to expansion-regime training | $> 0.5$ (TBD)       |
| $\lambda$ | Exponential decay for uncertainty targets                        | 5/6 (from KataGo)   |
| $d_s$     | Dimension of static latent $\mathbf{z}_s$                        | 32–64               |
| $r$       | Channel depth of action latent $\mathbf{z}_a$                    | 4–8 (or omitted)    |
| $d_e$     | Per-child embedding dimension                                    | 32–128              |
|           | Annealing schedule for LoTE/LoTV regularization                  | TBD                 |

Table 1: System hyperparameters.